

# AI Learning Path — Project Handoff

A complete summary of the project so it can be continued in a fresh chat. Paste this whole document, and attach the latest `learning-app.html` .

---

## 1. What this is

A **single self-contained** `learning-app.html` **file** — no backend, no build step, no dependencies except PDF.js from a CDN. Open it in a browser and it runs.

**Core loop:** the user uploads a PDF (or pastes text) → the app calls an AI model to extract 10-20 concepts and order them by prerequisite → it builds a Duolingo-style learning path → each node opens a multi-step interactive lesson grounded in the user's actual document.

**Who's building it:** a solo developer, not deeply technical, working from a Windows laptop (Chrome). Communicates in Hebrew. The long-term goal is a monetizable educational product, but the current artifact is a browser-only prototype.

---

## 2. Current architecture

Everything lives in ONE html file. Structure:

- `<style>` — all CSS. 3D path, lesson screen, question types, visuals, library.
- `<body>` — API-key modal, library screen, source picker (upload/paste tabs), learning-path screen, lesson screen (separate full-screen overlay), preview dialog, loading overlay.
- `<script>` — ~100 functions, all vanilla JS. No framework.

### AI provider

- **Anthropic Claude API, called directly from the browser.**
- Model: `claude-haiku-4-5` (constant `AI_MODEL` , ~line 1616) — cheapest, good enough. To upgrade quality, change this one string to `claude-sonnet-4-6` .
- Headers: `x-api-key` , `anthropic-version: 2023-06-01` , and `anthropic-dangerous-`

`direct-browser-access: true` (required for browser calls).

- The user's API key lives in `localStorage`, sent only to Anthropic.
- **We migrated providers twice during the build: Anthropic → Gemini → Groq → Anthropic.** The final choice is Anthropic. Any leftover mention of Gemini/Groq in old chat is stale.

## Storage model (`localStorage`)

Multi-course. Keys:

- `course_library` — array of `{id, title, language, conceptCount, created}`
- `active_course_id` — which course is currently open
- `course:<id>` — the full course object (concepts, `courseName`, language)
- `progress:<id>` — per-course progress `{lessonIndex: {completed, lesson, accuracy, xp}}`
- `source:<id>` — the raw source text for that course (used for grounding)
- `ai_usage` — cumulative token/cost tracking
- Legacy single-course keys ( `course_data` , `learning_progress` ) auto-migrate on load.

## Cost model (measured, not guessed)

- Haiku pricing: \$1/M input tokens, \$5/M output tokens.
- **Output is ~82% of a lesson's cost** (it's 5x the price of input).
- One lesson  $\approx$  \$0.01. A 15-lesson course  $\approx$  \$0.15. **\$5  $\approx$  34 full courses.**
- Lessons are **cached** after first generation — regenerating is free. This is the single biggest cost saver. A visible usage badge shows spend + calls + cache hits.

---

## 3. The lesson experience (what a lesson contains)

Each lesson is a **separate full-screen page** (not an inline panel), reached by tapping a path node, which first opens a **preview dialog** (concept name, why it matters, difficulty, and a tag showing whether the lesson is cached/free or will-generate/costs).

Lesson = a sequence of steps, built from whatever the model returned (missing pieces are skipped):

1. **Hook** — a curiosity-sparking fact, not a definition
2. **Prediction** — a guess-before-we-explain question (not graded)
3. **Concept cards** (3-5) — one idea each, 2-3 sentences, optional everyday analogy
4. **Worked example** — a problem with reveal-as-you-go steps
5. **Guided practice** — one question with an optional hint
6. **Quiz** (4-5 questions) — MIXED interaction types (see §4)
7. **Mini challenge** — a capstone combining several ideas
8. **Summary** — main idea, key facts, common mistake, real-world use
9. **Memory check** — user explains in their own words; AI evaluates
10. **Complete screen** — XP, accuracy, time, confetti

A progress bar tracks position in the lesson. Back button / Escape exits WITHOUT celebrating (this was a bug we fixed — see §6). “Continue” from the complete screen returns to the path, celebrates, and unlocks/scrolls to the next node.

## Visuals (auto-generated diagrams)

The model attaches a **structured spec** to cards where a diagram helps; the app renders it.

**No image generation, no extra API calls, deterministic.** Six types: `flow`, `compare`, `hierarchy`, `timeline`, `table`, `bar`. A malformed spec is dropped, never crashes the lesson. RTL-aware (arrows/timelines flip for Hebrew).

## Grounding in the source

This was a **major bug fix**. `generateLesson` originally received only a concept name + one-line description, so the model invented questions from general knowledge. Now: the source text is chunked (sentence-aware, `CHUNK_CHARS=1200`), and each lesson retrieves the passage its concept came from via **TF-IDF scoring** with a relevance floor. That excerpt (`EXCERPT_BUDGET=2400` chars) goes into the prompt with strict grounding rules: every fact/number/date must come from the source; distractors must be plausible misreadings of the source, not outside facts.

---

## 4. Question / interaction types

The quiz is NOT plain multiple-choice anymore. Seven **tap-based** interaction types (no drag-and-drop — it’s fiddly on mobile). The model picks the best type per concept and must

use  $\geq 3$  different types per lesson, never the same twice in a row.

- **choice** — one right answer among options
- **boolean** — true/false on a statement
- **order** — tap items to build a sequence (steps, chronology, size)
- **categorize** — tap an item, tap its bucket
- **blank** — fill a \_\_\_\_ gap by tapping the right word
- **match** — tap a term, tap its pair; colored line connects them
- **mistake** — find the one wrong statement among correct ones

Each type has a validator ( `normaliseQuestion` ), a renderer ( `renderQuestion` dispatches by type), and a grader ( `wireQuestion` dispatches by type). All grade to right/wrong + explanation. Wrong answers reveal the correct answer with an explanation.

---

## 5. The AI Tutor

A dock pinned to the bottom of the lesson screen, available throughout. Opens to **9 one-tap action chips** (plus a free-text box): explain simpler / go deeper / another example / real-world use / practice question / harder challenge / quiz me / summarise lesson / compare with another concept.

**Context-efficient:** the tutor sees only the CURRENT step (not the whole lesson, never the PDF) plus course position and the user's running accuracy — ~45 tokens. If accuracy drops below 50%, it's silently told to slow down and simplify.

---

## 6. Language handling

- The course is written in **whatever language the source material is in**. Upload a Hebrew PDF → Hebrew course. Spanish → Spanish. The model reports the language; there's a script-detection fallback (Hebrew, Arabic, Russian, Japanese, Chinese, Korean) if it doesn't.
- Every downstream prompt (lessons, tutor, feedback) is told to write in that language.
- **RTL support:** Hebrew/Arabic content is right-aligned, including diagrams (timeline, arrows flip). The UI chrome itself stays LTR/English.

- We fully removed the earlier English/Hebrew UI-toggle system — the interface is English, the *content* follows the source.
- 

## 7. Full build history (what we did, in order)

1. Built the initial MVP as one HTML file: upload → extract concepts → path → lessons.
2. Fought through API provider setup. Tried Gemini (key format confusion, then a 400 ), then Groq (worked, free), finally settled on **Anthropic** once the user bought \$5 of credits. Along the way learned: the console is at **platform.claude.com**, billing is buy-credits (not a monthly plan, but effectively pay-as-you-go with a ceiling).
3. Discovered `file://` CORS + Chrome caching issues — the user kept loading a STALE downloaded file. (Recurring theme: always delete old `learning-app*.html` from Downloads before re-downloading, and hard-refresh.)
4. Made lessons a **13-step Duolingo/Brilliant flow** instead of one question.
5. Made the learning path **3D** (perspective, lit nodes, press animations, particles).
6. Made the lesson a **separate full-screen page** with preserved path scroll position.
7. Added **cost optimization**: per-task token budgets, usage tracking, cache accounting.
8. Added **auto-generated visuals** (6 diagram types from structured specs).
9. Upgraded the **tutor** to 9 context-aware actions.
10. Removed Hebrew UI; made content-language follow the source; added RTL.
11. Added **multi-course library** (up to 8 courses) + **source grounding** (TF-IDF retrieval so questions come from the actual PDF).
12. Added **7 interactive question types** replacing plain multiple-choice.

### Bugs we found and fixed along the way

- **i18n self-reference** — the translation dict referenced `t()` before it existed, killing the whole script on load (looked like “the button doesn’t work”).
- **One-way upload door** — “use file instead” hid the paste box with no way back.
- **Confetti on exit** — leaving a lesson fired the completion celebration.
- **Lesson marked complete on load** — progress used truthiness of a cached lesson.

- **Infinite recursion** — `displayLearningPath` ↔ `loadLesson` .
  - **Truncated JSON discarded** — token budget too low for Hebrew (2x tokens); the paid-for response was thrown away. Fixed with higher budgets + a tolerant `extractJSON` that repairs/closes truncated JSON.
  - **Silent lesson failures** — empty responses, thinking-only responses, and husk lessons failed with NO message; the spinner vanished and nothing opened. Now every failure path reports. Broken cached lessons self-heal on next open.
  - **Stale-file confusion** — repeatedly, the real “bug” was Chrome serving the old file.
- 

## 8. What's LEFT (not yet built)

### Requested but not done (browser-buildable)

- **Real Boss lessons** — currently just larger lessons; should combine concepts from several previous lessons into one scenario.
- **Analytics dashboard** — per-topic mastery, strong/weak topics, study streak, weekly activity, recommended next/review lesson.
- **Spaced repetition** — promised in the original MVP, never built. Pure logic, no AI, cheap to add. Good candidate for “next thing.”
- **More interactive activities between cards** (currently practice is tap-only Q&A).
- **Gamification** — streaks, coins, badges, avatars, leaderboards.
- **Test mode, notes, highlighting, offline mode.**

### Requested but needs audio/precision the model lacks

- Click-the-correct-area-on-an-image (needs precise coordinates — model is bad at this)
- Listening / speaking / pronunciation (needs audio I/O)

### Requires a backend (impossible in a single HTML file)

- The entire subscription + credits system (users pay → you buy API usage from their fee)
- Model routing (expensive model for extraction, cheap for feedback)
- Admin cost dashboard, user accounts, cross-device sync
- Hiding the API key server-side (right now it's exposed in the browser — fine for

personal/local use, NOT for sharing the file with others)

**The productization path (discussed but not built):** build a small backend (Node/Vercel/Railway) holding ONE API key server-side; users hit your server, not Anthropic; you charge a subscription and pass through the API cost, keeping the margin. This is the right next step once the learning experience is finished.

---

## 9. Key constants (where to tune things)

- `AI_MODEL` — the model string. Swap to `claude-sonnet-4-6` for higher quality.
  - `MAX_TOKENS = { path: 4000, lesson: 5000, tutor: 400, feedback: 250 }` — output caps.
  - `MAX_COURSES = 8` — library limit (localStorage is ~5MB, source text dominates).
  - `CHUNK_CHARS = 1200`, `EXCERPT_BUDGET = 2400` — retrieval tuning.
  - `PRICE_IN / PRICE_OUT` — for the cost display; update if pricing changes.
- 

## 10. How to work on this file (important for the next session)

- It's ONE big HTML file. Edit in place.
- After ANY change: verify JS syntax ( `node --check` ), that every `getElementById` target exists in markup, that CSS braces balance, and that the whole script *executes* (not just parses) with a stubbed DOM. Several past bugs only appeared at load time, not at parse time.
- Test failure paths, not just the happy path — the silent-failure and truncated-JSON bugs were both invisible until every branch was exercised.
- The model (Haiku) drifts from the JSON schema. Everything that parses model output ( `extractJSON`, `normaliseLesson`, `normaliseQuestion`, `validVisual` ) must tolerate missing/malformed fields and drop rather than crash.
- **Always tell the user to delete old `learning-app*.html` from Downloads and hard-refresh** — stale files caused more “bugs” than real code did.